🧠 1. What is a Data Structure?

A **Data Structure (DS)** is a *way of organizing, managing, and storing data* so it can be used efficiently.

In Python, data structures are the backbone of programming logic and algorithms.

Think of them as containers or blueprints that define **how data is stored and accessed.**

📦 2. Classification of Data Structures

Type	Description	Examples
1. Built-in Data Structures	Already provided by Python	List, Tuple, Set, Dictionary
2. User-defined Data Structures	Built by programmers using classes or objects	Stack, Queue, Linked List, Tree, Graph

---

🧩 3. Built-in Data Structures in Python

Let's explore each one with examples 👇

3.1 List (Dynamic Array)

- Ordered
- Mutable (can be changed)
- Allows duplicates

Example:

```
# Creating a list
fruits = ["apple", "banana", "cherry"]


# Access elements
print(fruits[0])  # apple
```

```
# Add element
fruits.append("orange")

# Remove element
fruits.remove("banana")

# Iterate
for fruit in fruits:
    print(fruit)
```

✅ **Use case:** Dynamic collections like shopping cart items, records, etc.

3.2 Tuple (Immutable List)

- Ordered
- Immutable (cannot be changed)
- Allows duplicates

Example:

```
coordinates = (10, 20, 30)
print(coordinates[1])  # 20
```

✅ **Use case:** Fixed data, e.g., GPS coordinates or config values.

3.3 Set (Unordered Unique Collection)

- Unordered
- No duplicates
- Mutable

Example:

```
numbers = {1, 2, 3, 3, 2}
```

```
print(numbers)  # {1, 2, 3}


numbers.add(4)

numbers.remove(2)
```

✅ **Use case:** Storing unique elements like IDs, tags, or categories.

3.4 Dictionary (Key-Value Pair)

- Unordered
- Mutable
- Key must be unique

Example:

```
student = {"name": "Akhilesh", "age": 10, "grade": "4th"}

print(student["name"])  # Akhilesh


student["school"] = "Green Valley"
```

✅ **Use case:** Mapping data like student records or API responses.

🧱 4. User-Defined Data Structures

When built-in ones are not enough, we **design our own** using Python classes.

4.1 Stack (LIFO – Last In First Out)

Example Implementation:

```
class Stack:
    def __init__(self):
        self.items = []
```

```
    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()

    def peek(self):
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

# Usage
stack = Stack()
stack.push(10)
stack.push(20)
print(stack.pop())   # 20
```

✅ **Use case:** Undo operations, recursion tracking, expression evaluation.

4.2 Queue (FIFO – First In First Out)

Example:

```
from collections import deque

queue = deque()
```

```
queue.append("A")
queue.append("B")
queue.append("C")

print(queue.popleft())  # A
```

✅ **Use case:** Task scheduling, printer jobs, process queues.

4.3 Linked List

Concept: Each node stores data and a reference to the next node.

Example:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
```

```
            last = last.next
        last.next = new_node

    def display(self):
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")

# Usage
ll = LinkedList()
ll.append(10)
ll.append(20)
ll.append(30)
ll.display()
```

✅ **Use case:** Dynamic memory allocation, music playlists, undo lists.

4.4 Tree (Hierarchical Structure)

Example:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
root = Node(10)
root.left = Node(5)
root.right = Node(15)
```

✅ **Use case:** Databases, search trees (BST), file directories.

4.5 Graph (Network of Nodes)

Example:

```
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"]
}

for node in graph:
    print(node, "->", graph[node])
```

✅ **Use case:** Social networks, maps, routing, web link analysis.

⚙️ **5. Choosing the Right Data Structure**

Goal	Best Data Structure
Fast lookup by key	Dictionary
Maintain order	List
Ensure uniqueness	Set
LIFO processing	Stack

Goal	Best Data Structure
FIFO processing	Queue
Dynamic relationships	Graph
Hierarchical data	Tree

💡 6. Summary

Category	Data Structure	Key Property
Built-in	List	Ordered, Mutable
Built-in	Tuple	Ordered, Immutable
Built-in	Set	Unordered, Unique
Built-in	Dictionary	Key-Value Mapping
User-defined	Stack	LIFO
User-defined	Queue	FIFO
User-defined	Linked List	Dynamic Nodes
User-defined	Tree	Hierarchical
User-defined	Graph	Network

🧱 Data Structures in Python — Overview Table

Category	Data Structure	Description	Key Properties	Common Use Cases
🧩 **Built-in**	**List**	Ordered, mutable collection of items	- Indexed - Allows duplicates - Dynamic size	✅ Shopping cart items ✅ Maintaining ordered data ✅ Dynamic arrays

Category	Data Structure	Description	Key Properties	Common Use Cases
🧩 **Built-in**	**Tuple**	Ordered, immutable collection	- Indexed - Allows duplicates - Faster than list	✅ Storing API responses ✅ Storing coordinates (x, y, z) ✅ Returning multiple values from functions ✅ Configuration or constants
🧩 **Built-in**	**Set**	Unordered, mutable, unique items	- No duplicates - Fast membership testing	✅ Removing duplicates from data ✅ Tag systems ✅ Membership checks (in operations)
🧩 **Built-in**	**Dictionary**	Key–value pair mapping	- Unordered - Fast lookup by key - Mutable	✅ Storing user profiles (key → value) ✅ JSON-like data ✅ Counting word frequencies
⚙️ **User-defined**	**Stack (LIFO)**	Last In First Out structure	- Push/pop operations - Controlled access	✅ Undo/Redo feature ✅ Browser history ✅ Expression evaluation
⚙️ **User-defined**	**Queue (FIFO)**	First In First Out structure	- Enqueue/Dequeue - Controlled order	✅ Printer queue ✅ Task scheduling ✅ Message

Category	Data Structure	Description	Key Properties	Common Use Cases
				handling in chat apps
⚙️ **User-defined**	**Linked List**	Nodes connected in sequence	- Dynamic size - Sequential access	✅ Music playlist navigation ✅ Undo/redo operations ✅ Dynamic memory allocation
🌳 **User-defined**	**Tree**	Hierarchical node structure	- Root, children, leaf - Recursive traversal	✅ File system hierarchy ✅ Database indexing (BST, AVL) ✅ XML/JSON parsing
🌐 **User-defined**	**Graph**	Collection of interconnected nodes	- Nodes and edges - Directed or Undirected	✅ Social networks ✅ Road maps and routing ✅ Web crawler algorithms
⚡ **User-defined**	**Hash Table**	Key–value storage using hashing	- Constant-time lookup - Collision handling	✅ Caching ✅ Indexing ✅ Symbol tables in compilers
🧠 **Special DS**	**Heap**	Binary tree used for priority queues	- Min/Max property - Efficient access to smallest/largest	✅ Priority scheduling ✅ Dijkstra’s algorithm ✅ Heap sort

Category	Data Structure	Description	Key Properties	Common Use Cases
📊 **Special DS**	**Deque (Double-ended Queue)**	Queue supporting insertion/removal from both ends	- Fast operations on both ends - Flexible structure	✅ Browser history ✅ Palindrome checking ✅ Sliding window algorithms

---

🔍 Summary Insights

When to Use	Best Data Structure
Need ordered, changeable data	**List**
Need fixed, ordered data	**Tuple**
Need unique values	**Set**
Need key-value mapping	**Dictionary**
Need LIFO control	**Stack**
Need FIFO control	**Queue**
Need dynamic memory allocation	**Linked List**
Need hierarchical data	**Tree**
Need interconnected relationships	**Graph**
Need fast lookup using hash	**Hash Table / Dict**